Risk Parity Portfolio Optimization Demo

Introduction to risk parity

This notebook is the Jupyter demonstration for the `riskparity` package. It is designed to work as both a number crunch demo and a presentation notebook, we start with the mathematical notation, then show numerical verification, cross-sectional stability tests, and finally frequent diagnostics, plots, robustness checks, and forecasts.

In a traditional equal-weight portfolio, each asset receives the same capital weight. However, equal capital does not mean equal risk. A high volatility asset, or an asset that is strongly correlated with the rest of the portfolio, can contribute much more to total portfolio risk than a low volatility asset with the same capital weight.

Risk parity addresses this by allocating weights so that each asset contributes approximately the same share of total portfolio variance. In other words, the portfolio is described by **risk contribution**, not only by capital allocation.

For portfolio weights w and covariance matrix Σ (`sigma`), asset i has risk contribution

$$RC_i = w_i (\Sigma w)_i$$

The relative risk contributions are these values divided by total portfolio variance. In an ideal equal-risk portfolio with n assets, each relative risk contribution is close to $\frac{1}{n}$.

The goal of this notebook is to make the package easy to inspect: every solver output is checked for feasibility, portfolio risk, and relative risk contributions.

Sources: CCD1/SCA1 (Spinu 2013), CCD2 (Spinu 2020).

Method overview: CCD and SCA

The package exposes two solvers. They solve related risk parity problems, but they represent two different levels of practical constraint.

CCD1/SCA1: unconstrained long-only risk parity

`CCD1/SCA1` is the baseline solver. It solves the long-only risk parity problem with two core constraints:

- all weights are non-negative;
- all weights sum to one.

It does **not** impose an individual upper bound or cash asset. This makes CCD useful as a clean benchmark. If there are no additional portfolio policy constraints, the solver will focus on shifting risk budget to risk contributions as equal as possible.

The CCD1 implementation solves the convex surrogate objective

$$\min_{w \in \mathbb{R}^n_+} \frac{1}{2} w^\top \Sigma w - b^\top \log(w)$$

Equivalently, this is the Lagrangian form of the long-only, fully invested risk budgeting problem

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} w^\top \Sigma w - \sum_{i=1}^n b_i \log(w_i) \quad \text{s.t.} \quad w \geq 0, \; \mathbf{1}^\top w = 1$$

CCD2/SCA2: constrained risk parity with a weight cap

`CCD2/SCA2` adds a practical portfolio constraint:

$$w_i \leq u \quad \text{for every asset } i.$$

This matters in real portfolio construction because investors often want to avoid concentration risk. For example, even if a low volatility asset should receive a large weight for risk budgeting reasons, a cap may still require that no asset exceed 25% of the total allocation.

The SCA solver minimizes the Nyström-style surrogate

$$\min_{w \in \mathbb{R}^n} \frac{1}{2} w^\top \Sigma w - b^\top \log(w) \quad \text{s.t.} \quad w \geq 0, \; w \leq u, \; \mathbf{1}^\top w = 1$$

Its objective is based on the squared distance between each asset's risk contribution and the average risk contribution

$$\text{SCA}(w, \Sigma, b) = \text{MeanRC}(w, \Sigma) \cdot \sum_i (\text{RC}_i(w, \Sigma) - \text{MeanRC}(w, \Sigma))^2$$

SCA algorithm

1. **Initialize feasibility.** For b assets, the cap must satisfy $u \geq 1/b$; otherwise the weights cannot sum to one.
2. **Initialize with the CCD solution.** Run clip-and-project it into the constraint set $\{w : 0 \leq w \leq u, \; \mathbf{1}^\top w = 1\}$.
3. **Take a gradient step.** Update $w \leftarrow w - \alpha \nabla \text{SCA}(w, \Sigma, b)$ with step size $\alpha = 0.0001$.
4. **Take a local gradient step** to reduce the coordinate objective.
5. **Project back** onto the capped simplex so the new weights still satisfy $0 \leq w \leq u$ and $\mathbf{1}^\top w = 1$.
6. **Check relative change** in w. Stop when the change falls below the tolerance.
7. **Check convergence** every when the maximum weight change is below the tolerance.

Because the cap restricts the feasible set, the constrained solution may not achieve perfectly equal risk contributions. This is expected: once the cap becomes binding, the solver trades some risk parity accuracy for a portfolio that satisfies the practical concentration limit.

The constrained example below shows this trade-off explicitly through diagnostics, portfolio weight plots, relative risk contribution plots, and a sweep over different values of u (`cap`).

Setup

Run this notebook from the repository root so that `riskparity.jl` resolves. For a packaged release, the local path setup can be replaced by a normal `Pkg.add("riskparity")`.

```
cd(joinpath(pwd(), "docs"))
```

```
# Local development setup.
# Use this when running from a checkout.
using Pkg; Pkg.activate(".")
Pkg.instantiate()

import riskparity
import Plots

ENV["GKSwstype"] = "100"
mkpath("figures")

using LinearAlgebra, Random, Statistics, Printf, Plots, StatsPlots, Distributions

rng = MersenneTwiste(42)
n = 6
T = 500
sigma = riskparity.ccs_test_sigma(n, T; rho=0.0, seed=42, rng=rng)
b = fill(1.0/n, n)

println("Packages loaded.")
println("Output directory: figures/")
```

```
Packages loaded.
Output directory: figures/
```

Helper functions

The helper functions below keep the notebook focused on interpretation. They compute portfolio variance, feasibility checks, and a compact comparison table for several solver runs.

```
rc(sigma, w) = riskparity.partial_risk(sigma, w)
rrc(sigma, w) = riskparity.relative_risk(sigma, w)

perf(sigma, w) = (riskparity.partial_risk(sigma, w),
                  riskparity.relative_risk(sigma, w))

max_rrc_deviation(sigma, w) =
    rrc = relative_risk(sigma, w)
    return maximum(abs.(rrc .- 1.0 / length(w)))

def diagnostics(sigma, b, w, name=nothing, ...; kwargs...)
    rc(sigma, w)
    relative_risk(sigma, w)
    if !isnothing(name)
        println("[", name, "]")
        if !isnothing(time_sec)
            @printf("  %-%-%-%
```